Do Marketplace Listings Help LLM Agents Choose Skills? Auditing Listing Adequacy in LLM Agent Marketplaces

Anonymous ACL submission

Abstract

LLM agents increasingly choose external skills from marketplace listings that expose only a user-facing name and description. We study whether these listings provide enough information for users and agent routers to understand a skill before installation. We define *listing adequacy* and introduce an Information Completeness taxonomy for apparent omissions and overclaims. Using CLAWHUB—a large open-source agent skill registry—as a case study, we build an LLM-based screening auditor calibrated against listing-only expert judgments and triangulated with implementation-grounded and programmatic validation. The audit finds that most listings appear adequate, while a small subset appear to omit security-relevant or functional capabilities or overclaim their scope. Controlled routing experiments across three model families show modest aggregate disclosure lift (+0.9pp), with skill names often serving as dominant matching cues. In a curated OpenClaw name-blind setting, however, disclosure repair yields +8.1pp routing lift (95% CI [1.5, 14.7], p=0.015), exposing a description-dependent failure that aggregate results mask. Finally, selection experiments show that agents prefer some functionally similar skills over others, and that both descriptions and names influence these choices. Overall, skill listings should be treated as active inputs to agent retrieval and selection, not as passive metadata.

1 Introduction

LLM agents increasingly rely on external skills—web search, code execution, API integration—to extend their capabilities (Schick et al., 2023; Qin et al., 2024; Patil et al., 2023). In marketplace-style deployments, skill selection is mediated by user-visible natural-language listings: the name and description that appear before installation.

Unlike API signatures or benchmark tool descriptions, marketplace listings are author-written, unverified, and often sparse. This creates an understudied dependency: listings serve as inputs to retrieval, comparison, and selection, yet their quality is not systematically assessed. If a listing omits a capability, that capability may become difficult for routers to retrieve or select. If a listing overclaims, users install skills that cannot deliver. Strategically written listings may also bias selection.

Prior work has studied tool-calling accuracy (Schick et al., 2023), prompt injection (Greshake et al., 2023), agent safety (de Benedetti et al., 2024), and tool retrieval (Shi et al., 2025), but typically treats skill descriptions as fixed ground truth. We study the complementary question: are marketplace listings adequate as inputs to retrieval and selection, and what happens when they are not?

We make three contributions, using CLAWHUB as our primary case study with SKILLS.SH as a descriptive comparison point. *First*, we formalize *listing adequacy* for agent skill marketplaces and introduce an Information Completeness taxonomy that separates user-facing screening labels from implementation-grounded validation (§3–4). *Second*, we build and calibrate an LLM-based screening auditor, then apply it to a large marketplace snapshot where it classifies 95.0% of listings as adequate and flags a small minority for apparent omissions or overclaims. Routing experiments across three model families show modest aggregate disclosure lift (+0.9pp), but a name-blind evaluation isolates a larger description-dependent effect (+8.1pp; 95% CI [1.5, 14.7]; p=0.015) (§5). *Third*, we use position-rotated selection experiments, description swaps, and name/description ablations to show that agents exhibit stable preferences among functionally similar skills, with descriptions and names contributing separable signals (§6). To our knowledge, this is the first large-scale empirical study focused specifically on user-

visible listing adequacy in an LLM agent skill marketplace.

2 Background and Related Work

2.1 Tool-Augmented LLMs and Skill Selection

Modern LLM agents use tool-calling APIs to extend their capabilities (Schick et al., 2023; Qin et al., 2024; Patil et al., 2023). In marketplace-style deployments, the agent receives a set of skill descriptions and must select the appropriate skill for each user request. The skill's natural-language description thus becomes the primary interface between skill authors and the agent's routing logic, directly determining which skills are retrieved, compared, and ultimately invoked. Hsieh et al. (Hsieh et al., 2023) show that tool documentation quality directly affects zero-shot tool-use accuracy, underscoring the importance of description fidelity.

2.2 Description Quality and Selection Bias

Several lines of work study how tool descriptions affect LLM tool use. SkillReducer (Gao et al., 2026) addresses missing or verbose skill descriptions and proposes token-efficient compression. BiasBusters (Anonymous, 2026) demonstrates that LLMs exhibit systematic skill selection preferences influenced by description features. Compared to these works, we focus on descriptions that are *present but inadequate*—marketplace listings that may omit critical information or mislead users about a skill's true scope, affecting both retrieval and routing. We extend BiasBusters' methodology to a real marketplace corpus and analyze which specific description features correlate with selection.

2.3 Agent Safety and Tool-Calling Benchmarks

AgentDojo (de Benedetti et al., 2024), InjectAgent (Zhan et al., 2024), and Yang et al. (Yang et al., 2024) study skill-level attacks including prompt injection, backdoors, and data exfiltration, focusing on risks *after* a skill is selected. ToolBench (Qin et al., 2024), API-Bank (Li et al., 2023), and recent tool retrieval work (Shi et al., 2025) evaluate whether LLMs can invoke the correct tool across large collections. However, these benchmarks assume that skill descriptions are ground-truth and do not study what happens when descriptions are incomplete or misleading. Our work fills this gap by treating descriptions as *variables* rather than constants, and by focusing on the temporally prior attack surface: the skill *description* as a pre-installation trust interface that determines *which* skill is selected in the first place. This parallels work on description-to-permission fidelity in mobile app stores (Felt et al., 2011) and API ecosystem security audits (Liao et al., 2024), but targets the emerging LLM agent marketplace setting. Figure 1 summarizes our pipeline.

3 Listing Adequacy Framework

3.1 Problem Definition

Given a skill listing $l = (n, d)$, where n is the user-visible name and d is the user-visible description, *listing adequacy* asks whether l provides enough information for a reasonable user or agent router to infer the skill's likely scope before installation. We operationalize "reasonable user" as someone with general technical literacy but no access to the implementation: can they form correct expectations about the skill's capabilities, data access, and side effects from the listing alone? We distinguish this from implementation correctness: a listing can be inadequate even if the implementation is benign and functional.

Scope. Our audit evaluates user-visible listing adequacy, not implementation correctness or malicious intent. Labels are screening signals for human review; downstream experiments test effects in controlled settings and do not estimate deployment-wide rates.

Information Completeness taxonomy. We formalize listing adequacy via *Information Completeness* (IC), shown in Table 1.

Our auditor operates on user-facing definitions only (left column), inferring *likely* capabilities from the skill name and category. The implementation-grounded definitions (right column) require reading source code and are outside the scope of our ecosystem-wide audit; we provide a partial validation on 150 sampled skills (§4.1). An omission is labeled IC-1s when it concerns credentials, network access, file-system modification, shell execution, or similarly security-relevant operations; other omissions are labeled IC-1q.

3.2 Data: Two Agent Marketplaces

We study two production marketplaces:

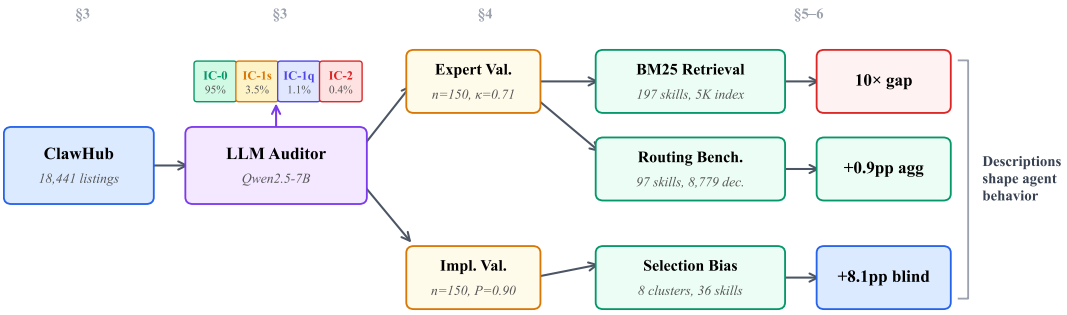


Figure 1: Listing-audit pipeline for the CLAWHUB snapshot (n=18,441 listings). The LLM auditor classifies listings along four IC axes, is calibrated against listing-only expert labels (n=150), and is checked with implementation-grounded validation (n=150). Downstream experiments measure retrieval, routing, and selection effects.

Class	User-facing	Implementation-grounded
IC-0	Description conveys the skill's likely scope	Description matches implementation capabilities
IC-1s	Description appears to omit security-relevant operations	Implementation contains undisclosed security-relevant operations
IC-1q	Description appears to omit functional capabilities	Implementation has features absent from description
IC-2	Description claims capabilities beyond likely scope	Implementation lacks claimed capabilities

Table 1: IC taxonomy: user-facing definitions (assessed by our auditor) vs. implementation-grounded definitions (requires implementation-level validation; see §4.1).

CLAWHUB. We use a commit-level snapshot of a large open-source agent skill platform. Each listing includes a user-visible name and optional description; a subset contains non-trivial implementation bodies used only for validation (§4.1). Our auditor processes the user-visible listing—the information visible before installation—not the implementation body.

SKILLS.SH. A curated registry aggregating skills from multiple platforms. Descriptions are substantially longer on average, providing more surface for both information and manipulation. We use SKILLS.SH for cross-platform comparison only. Table 2 summarizes the two snapshots.

3.3 Abstraction Gap

A simple lexical approach to detecting description–capability mismatches would compare code-level patterns (*e.g.*, `subprocess`, `os.environ`) against description keywords (*e.g.*, "execute shell command," "API key"). We identify a key limitation of this methodology.

	CLAWHUB	SKILLS.SH
Total skills	18,441	116,208
With description	13,576	114,515
Median desc length	159 chars	487 chars
With non-trivial impl. body	6,912	—

Table 2: Corpus statistics. Although every CLAWHUB entry contains a `SKILL.md` file, only a subset contain non-trivial implementation bodies suitable for implementation-grounded validation (§4.1).

Marketplace descriptions operate at a high semantic level. Traditional regex detectors flag a *vocabulary mismatch* between description and code: e.g., "publishes to WeChat" does not contain `subprocess.run()`, so the skill is flagged. In a pilot study, such flags were overwhelmingly false alarms: the description was simply operating at a higher semantic level. This *abstraction gap* can inflate apparent mismatch rates by more than an order of magnitude (Figure 2). The gap shows that listing adequacy cannot be evaluated by lexical overlap between descriptions and implementation tokens; the relevant question is whether the listing conveys the capability at the level users and agents operate.

3.4 LLM-Based Listing Auditor

We replace heuristic detectors with an LLM-based auditor that reduces false positives from lexical mismatch: it understands that "publishes to WeChat" implies credential handling without requiring the description to mention `api_key` verbatim.

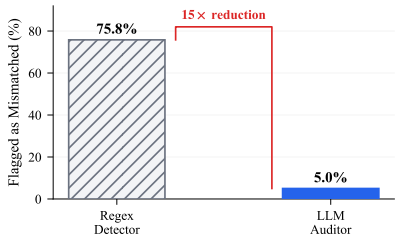


Figure 2: Abstraction gap: lexical detectors conflate vocabulary mismatch with genuine omission, while semantic auditing reasons at the user's level.

For each skill, the auditor receives the marketplace description and classifies along four axes: (1) IC class (IC-0/IC-1s/IC-1q/IC-2); (2) overclaim severity (none/minor/major); (3) quality issues (Q1–Q4: missing parameters, vague triggers, functional overclaim, missing permissions); (4) abuse signals (A1–A3: impersonation, concealment, inflation).

The prompt template instructs the auditor to reason as a *reasonable marketplace user*:

> "You are an auditor for an AI agent skill marketplace. Given ONLY the skill name and description (what a user sees before installation), assess whether the description adequately represents the skill's likely capabilities."

The auditor responds with a structured JSON object containing the IC class, severity, reasons, and confidence score. We use a structured-rationale prompt that asks the model to summarize expected capabilities, identify potential gaps, and output a JSON classification.

Model. We use Qwen2.5-7B-Instruct (Team, 2025), served via HuggingFace Transformers on a single NVIDIA A100 GPU. Temperature is set to 0.1 for near-deterministic outputs; top-p/top-k are not overridden from serving defaults, inputs are capped at 6k characters, and the full prompt template is provided in Appendix A. The choice of an open-weight model improves reproducibility.

Scope. The auditor processes only user-visible listing fields—the skill name, category, and description visible on the marketplace before installation. This represents the "reasonable user" perspective: can a person reading only the marketplace listing understand what the skill will do? We deliberately exclude the `SKILL.md` implementation file (which users do not typically read before installation) to avoid conflating developer documentation with user-facing information. We acknowledge a tension: the auditor infers *likely* capabilities from the skill name, which may produce false positives (e.g., inferring network access for a "database connector" that only manages local schemas). Our validation (Table 3) shows this manifests as moderate precision for flagged classes, acceptable for a screening tool but not a ground-truth oracle.

What the audit does not claim. The audit does not determine whether a skill is malicious or whether its implementation is incorrect. Auditor labels are screening signals, not final security judgments.

Throughput. The audit can be run overnight on a single modern GPU, making the approach practical as a publish-time screening step for marketplace operators.

3.5 Human Validation

To calibrate the auditor, two annotators with experience in agent-tool systems independently labeled a stratified sample drawn according to auditor-predicted class. Annotators were blind to auditor predictions and received only the skill name, description, and category—the same information available to the auditor. A pilot round was used to calibrate the annotation guideline (reproduced in Appendix J) before the main labeling. Disagreements were resolved by discussion to a consensus label.

Important scope note: because annotators did not have access to skill implementations, this validation measures **auditor–human agreement under the same incomplete-information condition** (listing only), not auditor correctness against implementation ground truth. Table 3 should therefore be read as a calibration check rather than an accuracy benchmark.

Agreement was substantial between annotators (Cohen's κ=0.71) and between the auditor and the consensus labels. Table 3 reports class-wise precision and recall.

The auditor is most reliable when identifying adequate listings, while flagged classes are noisier. This is appropriate for screening: flagged listings should receive human review rather than automatic enforcement.

IC-0 false-negative estimation. Among 50 auditor-labeled IC-0 skills in the implementation-grounded validation, all 50 were classified as ad-

IC Class	n	Precision	Recall
IC-0	50	0.97 [.88,.99]	0.95 [.85,.98]
IC-1s	50	0.72 [.58,.83]	0.80 [.67,.89]
IC-1q	25	0.68 [.48,.83]	0.64 [.45,.80]
IC-2	25	0.76 [.57,.89]	0.72 [.52,.86]

Table 3: Auditor–expert agreement under the listing-only condition (same information to both), with 95% Wilson confidence intervals. IC-0 agreement is high; flagged classes show moderate precision, consistent with screening-tool expectations. Full annotation guidelines and borderline examples in Appendix J.

IC Class	N	%
IC-0 (adequate)	17,526	95.0
IC-1s (apparent sec. omission)	643	3.5
IC-1q (apparent func. omission)	196	1.1
IC-2 (apparent overclaim)	72	0.4
Unparseable	4	<0.1
Total	18,441	100

Table 4: Auditor-estimated listing adequacy for the full CLAWHUB snapshot (n=18,441 listings), calibrated against listing-only expert labels in Table 3. Corpus-level prevalence may differ because validation was stratified (see §Limitations).

equate by LLM-derived implementation inspection (0 false negatives; Wilson 95% CI upper bound: 7.1%). Among sampled implementation-inspectable IC-0 skills, this suggests the auditor's IC-0 label is conservative. Extrapolating to the full corpus should be done cautiously: the upper bound of approximately 1,250 listings among 17,526 IC-0 entries assumes the inspectable sample is representative, and a larger random sample with human code audit would narrow this estimate.

4 Ecosystem-Wide Audit

We apply the auditor to the full CLAWHUB snapshot. Table 4 summarizes the resulting screening labels.

Observation 1 ***Most listings appear adequate under auditor screening.*** *The large majority of listings are classified IC-0, and stratified validation shows that this class is reliable. In contrast, regex-based detection substantially overstates mismatch because it confuses abstraction with omission. Among no-description listings, the auditor still classifies 88.2% as IC-0 (Table 15) because the skill* name *alone often conveys sufficient scope (e.g.,* `gmail`, `web-search`*). This reflects the taxonomy design: IC-0 requires only that a reasonable user can form correct expectations from the listing as a whole, not that every field be populated.*

IC Class	None	Minor	Major
IC-0	17,449	77	0
IC-1s	0	547	96
IC-1q	0	170	26
IC-2	0	61	11
Total	17,449	855	133

Table 5: Joint distribution of IC class and maximum severity per skill. Major-severity IC-1s listings are the highest-priority audit targets.

Observation 2 ***A small subset is flagged for apparent omissions.*** *The auditor flags a small minority of listings for apparent security-relevant or functional omissions. These are screening rates rather than final prevalence estimates; we partially validate them against implementation bodies in §4.1.*

Cross-platform context. SKILLS.SH descriptions are longer than CLAWHUB descriptions, giving authors more room for both useful detail and manipulative text. This supports a hypothesis about length and adversarial surface area, but does not establish causality.

Severity distribution. Table 5 shows the joint distribution of IC class and maximum auditor-assigned severity. Major-severity IC-1s listings are the highest-priority targets for human review because they appear to omit security-relevant operations.

Qualitative examples. IC-1s listings span diverse categories: e.g., a messaging skill not mentioning credential handling, or a data-access skill not mentioning network access. IC-2 listings include overclaimed capabilities such as "security scanning" that describes only API wrapping.

4.1 Implementation-Grounded Validation

To partially address the gap between listing-level and implementation-level assessment, we sample skills stratified by inferred IC class and compare auditor classifications against LLM-assisted capability extraction from `SKILL.md` implementation bodies.

For each skill, an independent LLM (separate from the listing auditor) reads the full `SKILL.md` implementation body and extracts implementation-level capabilities—shell execu-

tion, credential access, network calls, and filesystem modification. The implementation annotator is blinded to the listing auditor's IC label. We then compare: (1) the listing auditor's IC class (based on name+description only) against (2) the implementation-derived label.

Results. Implementation-grounded validation supports the main screening result: most auditor-flagged labels are also supported when the implementation body is inspected. The false positives mainly arise when a skill name implies broader scope than the implementation delivers, consistent with the inherent tension of inferring capabilities from listings alone. The full confusion matrix is reported in Appendix G.

Caveat: this validation uses LLM-based capability extraction from `SKILL.md` text, not manual code audit. The verification chain is therefore *LLM auditor → LLM implementation extractor*, with no human-verified ground truth in the loop. The programmatic audit below partially addresses this gap, but a larger manual audit would further strengthen these estimates.

4.2 Programmatic Capability Verification

To provide lower-bound supporting evidence independent of any LLM, we run a regex-based capability detector over the raw implementation files of 67 IC-1s skills from our routing benchmark. Unlike the abstraction-gap analysis in §3.3, which showed that regex overstates mismatches at the corpus level, here we use regex conservatively: we only count a capability as *supported* when the code contains direct evidence (e.g., literal `subprocess` calls), accepting that we will miss capabilities expressed through abstractions. For each of five capability types—shell execution, credential access, network access, filesystem operations, and email—we search for concrete code-level indicators (e.g., `subprocess`, `API_KEY`, `requests.`, `open(`, `smtp`). We compare detected capabilities against description mentions to identify programmatically supported omissions.

Table 6 shows that 66.7% of flagged omissions are supported by implementation evidence, and 76% of skills (51/67) have at least one programmatically supported omission. Shell execution and credential access show high precision (75–85%), while filesystem detection is noisier due to broad pattern matching. These results are conservative: regex detection misses capabilities expressed through framework abstractions (e.g., a skill using a database ORM rather than raw `open()` calls).

Capability	Flagged	Supported	Support rate
Shell execution	53	45	84.9%
Credential access	28	21	75.0%
Network access	35	17	48.6%
Filesystem	19	4	21.1%
Email	9	9	100%
Overall	**144**	**96**	**66.7%**

Table 6: Programmatic verification of auditor-flagged capability omissions (67 IC-1s skills). "Flagged" = description lacks mention but name or context suggests the capability; "Supported" = implementation code contains concrete evidence. Overall support rate is 66.7%; 76% of skills have $\geq$1 programmatically supported omission.

5 Impact on Retrieval and Routing

5.1 Lexical Discoverability Experiment

Setup. We measure how description sparsity affects lexical discoverability. We sample skills with short descriptions and substantive implementation bodies, which provide a stress-test set for vocabulary mismatch, and embed them in a large BM25 index. For each skill we generate *goal* queries (*e.g.,* "I need to manage database connections") and *scenario* queries (*e.g.,* "My system requires automated report generation") from body vocabulary, measuring Recall@10 under original vs. augmented descriptions (body-derived disclosure sentences added). Statistical significance is assessed separately for goal and scenario queries using paired tests; implementation details are provided in Appendix B. We use BM25Okapi (Robertson and Zaragoza, 2009) with default parameters ($k_1 = 1.5, b = 0.75$).

Design note: Queries are drawn from body vocabulary, so the measured gain is a lexical upper bound; real user queries may yield smaller gains. Table 7 reports the results.

Observation 3 ***Sparse listings reduce lexical discoverability.*** *Augmenting listings with capability vocabulary improves Recall@10 in this stress test, especially for goal-style queries. Because queries are generated from implementation vocabulary, the result should be read as an upper bound on real-world lexical gains.*

Query type	Original	Augmented	p
Goal	58.5%	65.1% (+6.6pp)	<0.001
Scenario	72.1%	75.0% (+2.9pp)	0.013

Table 7: BM25 Recall@10 under original vs. augmented listings (n=197 skills embedded in a 5K-skill index). Deltas are +6.6pp (95% bootstrap CI [1.0, 12.4]) for goal queries and +2.9pp (CI [−2.2, 7.8]) for scenario queries. Query generation makes this a lexical stress test rather than a deployment prevalence estimate.

Router	Condition	Original	Disclosed	Length ctl	Structured
OpenClaw	Accuracy	82.5	**83.5**	82.5	82.6
GPT-5.4-mini	Accuracy	83.7	83.7	82.2	82.8
Gemini 2.5 Flash	Accuracy	75.8	77.6	76.0	78.0
Mean disclosure lift					**+0.9pp** (range 0.0 to +1.8)

Table 8: Overall routing accuracy (%) by condition and router (97 target skills: 78 IC-1 plus 19 IC-0 controls; 8,779 valid decisions across 3 routers). Targeted disclosure changes overall accuracy by +1.0pp for OpenClaw, +0.0pp for GPT-5.4-mini, and +1.8pp for Gemini; these aggregate effects are descriptive, with the pre-specified name-blind contrast tested in Table 9.

5.2 Routing Repair Experiment

Research question. Do IC-1 capability–description mismatches cause measurable routing failures, and do targeted disclosure repairs close them?

Experimental design. We construct a routing benchmark at two scales. A small pilot tests feasibility; a larger replication tests generalizability across skill types, description conditions, and router models.

For each skill, we generate four query types. *Description-mentioned* queries reference capabilities explicitly stated in the listing. *Undisclosed-capability* queries reference capabilities present in the implementation but absent from the listing. *Name-blind* queries describe undisclosed capabilities without any vocabulary related to the skill name, isolating the description signal from name-based matching. *Unsupported-capability* queries target capabilities the skill does not have (expected rejection). Each query is evaluated under 4 description conditions:

1. **Original**: the marketplace description as-is.
2. **Targeted disclosure**: one sentence appended disclosing the omitted capability (*e.g.,* "Requires an API key and can read local files").
3. **Length-matched control**: one sentence of equal length but irrelevant content (*e.g.,* "Plan carefully and review changes early"), isolating length from semantic contribution.
4. **Structured repair**: a Capabilities/Permissions/Limitations template.

The large-scale benchmark contains 97 target skills from the CLAWHUB corpus: 78 auditor-flagged IC-1 skills (IC-1s/IC-1q) and 19 IC-0 controls, selected using manifest-declared permissions cross-referenced against description text. The IC-1 subset covers apparent security and functional omissions; the IC-0 controls prevent the benchmark from containing only omission-positive targets. Skills are stratified across common omission types such as shell access, API keys, file-system access, browser automation, OAuth, email, and collaboration platforms. Each routing prompt places the target skill among distractors drawn from popular skills, using description-only presentation with deterministic target positioning for reproducibility. We evaluate three routers—OpenClaw (default gateway), GPT-5.4-mini, and Gemini 2.5 Flash—to test whether findings generalize across model families.

Pilot results. The pilot shows the expected asymmetry: queries that match the listing route reliably, while queries that depend on omitted capability vocabulary are much more fragile. Disclosure repair removes this gap in the pilot setting, motivating the larger benchmark below.

Large-scale results. Table 8 reports routing accuracy across all 4 conditions and 3 routers.

Across all three routers, overall positive-query accuracy is high (75.8–83.7%). For description-trigger queries, original-condition recall is 94.4%, suggesting that stated capabilities route reliably in this benchmark. Disclosure lift is modest at the aggregate level because *informative listing text already suffices for most skills*: 67/78 IC-1 target skills (85.9%) are routed correctly by a majority of original-condition decisions. The isolated description-dependent impact of repair is masked by this high baseline.

Name-blind analysis. To isolate the description signal from name-based matching, we construct *name-blind* queries that describe undisclosed capabilities without referencing the skill name or its synonyms (43 IC-1 skills, 516 routing decisions). Table 9 shows the key result.

The disclosure lift is about 6.8× larger for

Condition	Desc-trigger	Body-trigger	Name-bl
Original	90.7	89.5	90.7
Targeted disclosure	90.7	90.7	**98.8**
Disclosure Δ	0.0	+1.2	**+8.1**

Table 9: Routing recall (%) by query type and condition (43 IC-1 skills, OpenClaw router). When queries avoid skill-name vocabulary (name-blind), the disclosure lift is **+8.1pp**—about 6.8× the name-containing body-trigger lift (+1.2pp). This indicates that skill names mask the routing deficit caused by incomplete descriptions.

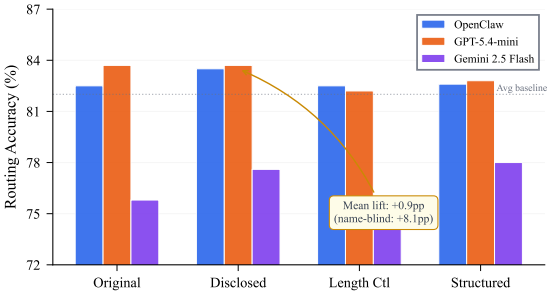


Figure 3: Routing accuracy across 4 description conditions and 3 routers. Targeted disclosure yields a modest aggregate lift, but the effect is masked by skill-name signals—see Table 9 for the name-controlled analysis.

name-blind queries (+8.1pp; 95% CI [1.5, 14.7]; p=0.015, n=86 per condition) than for standard body-trigger queries (+1.2pp), and 9× larger than the aggregate result (+0.9pp). In the large-scale name-avoidant benchmark across all three routers, mean lift is +1.1pp (OpenClaw −1.3pp, GPT-5.4-mini +1.9pp, Gemini 2.5 Flash +2.8pp) (Appendix F), directionally consistent but smaller than the curated name-blind result, likely because the large-scale queries include skills with moderately informative names. For the 4 skills with fully opaque names (n=4; *e.g.,* `nano-banana-pro`), name-blind recall jumps from 0% to 100% under disclosure—an illustrative (but small-sample) complete recovery.

Observation 4 ***Skill names dominate routing; description gaps are masked until name signals are removed.*** *Aggregate disclosure lift (+0.9pp) underestimates the isolated description-dependent impact of repair. When name-based matching is controlled via name-blind queries, the lift rises to +8.1pp, and skills with opaque names show 0%→100% recovery.*

Query constructedness caveat. All queries in both the aggregate and name-blind benchmarks are synthetically generated from implementation vocabulary or repair sentences. This design is appropriate as a diagnostic stress test: it measures the *diagnostic upper bound* of the routing impact of description gaps. However, it may overestimate effects relative to natural user queries, which may not precisely target omitted capabilities. Future work should validate with externally authored task descriptions. Figure 3 summarizes the aggregate routing results across all conditions.

Comparison with SkillReducer. SkillReducer (Gao et al., 2026) proposes description *compression* to reduce token costs. Our routing benchmark (Tables 8 and 9) reveals a complementary problem: IC-1 descriptions can omit capability vocabulary that routing depends on. Critically, this gap is *masked* at the aggregate level by skill names that serve as strong routing cues; it emerges clearly only in the name-blind evaluation. Losslessly compressing an incomplete description preserves the same routing failure—compression cannot repair a semantic gap. Description repair (augmentation) and compression are orthogonal interventions that should be applied sequentially.

6 Selection Bias in Listings

This section asks whether agents show stable skill preferences, which listing features correlate with selection, and whether descriptions and names provide separable signals.

6.1 Controlled Selection Benchmark

Setup. Following BiasBusters (Anonymous, 2026), we curate clusters of functionally similar skills from ClawHub and ask LLM agents to choose among them under rotated presentation orders. This lets us distinguish skill-level preference from position bias. We construct multiple clusters and evaluate them with cyclic position rotations across models; full cluster sizes, model configurations, and call counts are reported in Appendix C. We measure two concentration scores: *skill-level concentration* (δ_{skill}) captures whether the model repeatedly selects the same skill, while *position-level concentration* (δ_{pos}) captures whether it repeatedly selects the same list position. A score of 0 means selections are perfectly uniform; a score near 0.8 means one

Cluster	δ_{skill}	δ_{pos}	Top-1 (%)
github_mgmt	0.800	0.000	GitHub (100)
image_gen	0.800	0.000	Flux Image (100)
video_gen	0.800	0.000	Google Veo (100)
memory_sys	0.800	0.137	Cognitive Mem. (100)
speech_to_txt	0.600	0.000	Speech To Text (80)
twitter_x	0.552	0.076	X Articles (71)
text_to_speech	0.400	0.000	Mac TTS (40)
web_search	0.400	0.000	Tavily (40)
Mean	**0.644**	**0.027**	—

Table 10: Selection concentration by cluster under cyclic position rotation. Skill-level concentration (δ_{skill}) is consistently higher than position-level concentration (δ_{pos}), suggesting that list order is not the main source of preference. Experimental details are reported in Appendix C.

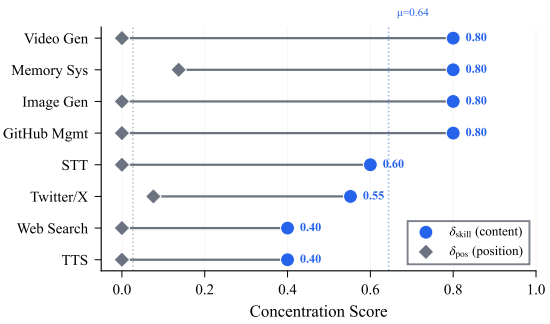


Figure 4: Selection concentration by cluster. Skill-level concentration (δ_{skill}, blue) is consistently high across clusters, while position-level concentration (δ_{pos}, gray) is near zero, indicating content-driven preference.

skill receives nearly all selections. Formally,

$$\delta_{\text{skill}} = \frac{1}{2} \sum_{i=1}^{K} |p_i - 1/K|$$

where p_i is the selection proportion of skill i and K is the cluster size; δ_{pos} is defined analogously over presentation positions. Table 10 reports results across all clusters.

Observation 5 ***Agents show stable skill preferences after position rotation.*** *Skill-level concentration is consistently higher than position-level concentration, indicating that preferences are not primarily an artifact of list order.*

Figure 4 visualizes these concentration scores across all clusters.

6.2 Exploratory Description Feature Analysis

Feature	r	Interpretation
Specificity	+0.654	Concrete details help
Readability (FK)	−0.526	Simpler text helps
Mentions AI/LLM	−0.397	AI keyword hurts
Name length	−0.340	Shorter names help
Promo word count	−0.090	Not predictive

Table 11: Pearson correlation between description features and selection rate (n=36 skills, R^2=0.52 for combined exploratory linear regression). *Caveat:* the sample is small and correlations are exploratory; they should not be interpreted causally. Leave-one-cluster-out analysis preserves the sign of the top two features; because the analysis is exploratory and cluster-confounded, we do not use these correlations for confirmatory hypothesis tests.

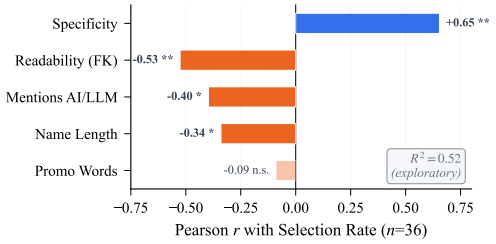


Figure 5: Pearson correlation between description features and selection rate. Specificity is the strongest positive predictor; promotional language is not predictive. The analysis is exploratory and should not be read causally.

Figure 5 visualizes these correlations.

Observation 6 ***Concrete and readable descriptions correlate with higher selection rates (exploratory).*** *The strongest correlates of selection are specificity and readability; promotional language shows little benefit (Table 11). These correlations are exploratory and may be confounded by cluster, brand, or capability type; the causal evidence comes from the swap and ablation experiments below.*

Preferred vs. excluded skills. Top-selected skills share concrete capability language, shorter names, and simpler descriptions. Excluded skills exhibit vague or circular descriptions.

Qualitative patterns. Manual inspection shows that top-selected skills use concrete action verbs and quantitative details, while bottom-selected skills use circular definitions or promotional language without specifics.

6.3 Automated Description Repair

While BiasBusters (Anonymous, 2026) proposes output-side mitigation, we test *input-side repair*: rewriting descriptions to start with concrete verbs, include quantitative details, and simplify language (full examples in Appendix H).

Cluster	Gemini 3.1 Pro		GPT-5.4-mini	
	Base	Swap	Base	Swap
web_search	Tavily	BraveApi*	Tavily	BraveApi (76%)
image_gen	Flux Img	falimagegen*	Flux Img	Flux Img†
video_gen	Google Veo	—	AI Vid	Sora (79%)
tts	Mac TTS	—	ElevenL	openai-tts (100%)
stt	Speech TT	—	EL STT	Voicenotes (68%)
github_mgmt	GitHub	—	GitHub	GitHub†
music_gen	music-cog	—	music-cog	Song (68%)
notion_mgmt	Notion API	—	Notion A.	Notion Enh. (96%)
Overall pattern			Most clusters reverse	

Table 12: Description-swap ablation: swapping descriptions often transfers selection preference, with exceptions in brand-dominant clusters. *Gemini reversal in tested clusters. †Brand-dominant exceptions. "Base" = most-selected skill before swap.

Cluster	Name-Anon	Name-Only	Desc-Only
web_search	.400	.400	.400
image_gen	.800	.800	.800
video_gen	.800	.800	.800
tts	.800	.800	.800
stt	.400	.800	.800
github_mgmt	.800	.800	.760
music_gen	.600	.800	.600
notion_mgmt	.560	.600	.440
Mean	**.645**	**.750**	**.675**

Table 13: Active ablation: selection concentration under name-anonymized, name-only, and description-only conditions. Description alone produces concentration in every cluster; in most clusters, NAME-ANON and NAME-ONLY select *different* winners, providing evidence for independent contributions.

Live validation. In the *description-swap* condition, we exchange descriptions between the most- and least-selected skills; in the *repaired-description* condition, we replace all descriptions with our repairs. We run the description-swap test across clusters on two models to test cross-model generality.

Across both models, the description-swap condition causes the originally dominant skill to lose all or most selection share (Table 12). Most clusters show a clear reversal; the main exceptions are *brand-dominant* skills (e.g., "Flux Image," "GitHub") where the name itself can dominate description content. Notably, the cross-model test reveals *model-specific brand preferences*: each model favors skills from its own ecosystem.

Under the repaired-description condition, selection concentration drops and previously invisible skills gain meaningful selection share.

Observation 7 ***Description swaps affect selection across model families, while brand effects vary by model.*** *Description swaps transfer selection preference across models, providing evidence that descriptions can influence selection in this controlled setting. However, each model shows distinct brand preferences (Table 12), suggesting brand-recognition patterns that resist description-level intervention.*

Name vs. Description Decomposition. To disentangle name and description contributions, we run three additional conditions: NAME-ANON replaces all names with `Tool A/B/C/D/E` while preserving descriptions; NAME-ONLY keeps real names but replaces all descriptions with a uniform template; DESC-ONLY combines anonymous names with real descriptions.

Table 13 reveals that both name and description independently affect concentration: NAME-ANON preserves concentration, while NAME-ONLY also produces strong concentration. Crucially, in most clusters the anonymized-name winner differs from the name-only winner (e.g., image_gen selects Tool C by description but Nanobanana by name), indicating that description content and skill identity exert *separable* effects on selection. Combined with the description-swap ablation, this suggests that description is an important—but not sole—contributor to selection, with brand-name recognition providing an additive boost.

Observation 8 ***Descriptions and names provide separable selection signals.*** *Anonymizing names preserves concentration and uniform descriptions still produce concentration via name recognition. In most clusters, the two signals select* different *skills, making it unlikely that either factor alone explains selection patterns.*

Brand and keyword effects. Supplementary experiments (Appendix I) test whether promotional keywords or brand-like name prefixes can shift selection. Keyword injection has little effect in our tests, while brand-like names substantially shift selection in several clusters—suggesting that name-level verification matters more than keyword filtering for marketplace review and validation.

7 Discussion

Listings as active inputs. Our results suggest that marketplace listings are not passive metadata: they are used as inputs to retrieval, comparison, and selection. Descriptions determine which skills are retrieved and compared; names carry brand-like signals that can dominate description cues.

For marketplace operators, the practical implication is not to expose raw IC labels to users, but to use them as internal screening signals paired with human-reviewed capability disclosures.

Semantic audit vs. lexical matching. The abstraction gap (Figure 2) shows that keyword overlap between high-level descriptions and low-level implementation tokens is not a meaningful adequacy metric. This parallels findings in summarization evaluation where n-gram overlap (ROUGE) poorly correlates with semantic faithfulness (Fabbri et al., 2021). LLM-based semantic auditing can distinguish genuine omission from normal abstraction—a capability for which regex detectors are poorly suited.

Disclosure as governance. Our routing experiments reveal an important confound: skill names provide such a strong routing signal that aggregate disclosure lift (+0.9pp) masks the isolated description-dependent impact. Name-blind analysis (Table 9) shows that disclosure lift reaches +8.1pp when name signals are controlled, and skills with opaque names recover from 0% to 100% routing accuracy. This has practical implications: in our benchmark, descriptive names often compensate for incomplete descriptions, but opaque or brand-like names make description quality decisive for routing. Input-side repair complements output-side filters (Anonymous, 2026): rather than changing selection after the fact, repairs improve the information available to the router before selection. For deployment, such repair should feed into structured permission fields and human review, not standalone automatic rewriting.

Security implications. Our adversarial experiments reveal an asymmetric threat landscape: promotional keywords have little effect on selection, while brand-like names can substantially redirect routing. This suggests that marketplace defenses should prioritize name-level verification and brand validation at publish time over keyword filtering.

8 Conclusion

We studied marketplace listing adequacy as an input to retrieval and selection in LLM agent ecosystems. Our audit suggests that most user-visible skill listings are adequate under a screening perspective, but a small subset contain apparent omissions or overclaims that warrant human review. Controlled routing experiments across three model families show that these gaps can affect downstream agent behavior, but the effect is largely masked by skill names that serve as dominant routing cues. A curated OpenClaw name-blind evaluation reveals that disclosure repair yields +8.1pp routing lift—indicating that descriptions can be critical for skills whose names do not convey their capabilities.

Our selection experiments further show that agents do not treat functionally similar skills uniformly: descriptions and names both shape choice as separable factors, suggesting that listings are active inputs to agent behavior rather than passive metadata. Future work should calibrate corpus-level prevalence with random human samples and test generalization across other agent ecosystems.

Limitations

Auditor scope. Our LLM-based audit operates on listings only—the "reasonable user" perspective. Skills whose documentation clarifies capabilities not in the marketplace listing may be classified as IC-1 when the full documentation is adequate.

Auditor model diversity. Our primary audit uses a single model (Qwen2.5-7B-Instruct). A cross-model check reveals divergence in IC-class assignment, suggesting that screening rates are model-dependent.

Name-inference conflation. Because the auditor infers "likely" capabilities from skill names, an IC-1 label may reflect overbroad name inference rather than a true implementation omission. Our implementation-grounded validation (Table 17) partially addresses this—but a finer-grained taxonomy separating "name-induced expectation" from "confirmed omission" would further strengthen the analysis.

Selection concentration coverage. The selection benchmark covers a limited set of functionally similar skill clusters. Directions are consistent and cross-model replicated, but more clusters would further stabilize feature-analysis magnitudes.

Routing experiment scale. The routing benchmark covers 97 target skills (78 IC-1 plus 19 IC-0 controls) across three router models (8,779 valid decisions), and the name-blind analysis covers 43 IC-1 skills (516 decisions). Results are limited to one marketplace and three router configurations; additional routers and platforms would strengthen generalizability.

Platform specificity. Results are measured on CLAWHUB and SKILLS.SH. Other agent platforms (LangChain Hub, GPT Store) may exhibit different description patterns.

Name and vendor confounds. Our experiment controls for position effects via cyclic rotations and for name effects via active name-anonymization ablation (Table 13). The ablation provides evidence that both name and description independently affect concentration, but this decomposition is tested on a single model (GPT-5.4-mini); cross-model replication would further strengthen the finding.

Ethics Statement

Our audit uses only publicly available marketplace data. Case studies use real skill names and descriptions from the public index; however, we report aggregate IC distribution statistics without identifying specific IC-1s skills by name to avoid reputational harm from auditor false positives.

No adversarial skills were deployed to any marketplace. We will coordinate disclosure of IC-1s findings with platform operators before publication, providing skill maintainers an opportunity to review, appeal, or correct their descriptions before any public release. We will not release skill-level IC-1s labels until maintainers have been notified and given an opportunity to respond. Public artifacts will use anonymized or hashed skill identifiers for flagged cases unless maintainers consent to disclosure.

Because auditor precision is imperfect for flagged classes, some flagged skills may be false positives. We therefore treat auditor labels as screening signals, not final judgments, and recommend platform operators verify flagged skills via human review before taking action.

Artifact release policy. We will release: anonymized IC distribution statistics, auditor prompts, query generation templates, routing benchmark code, and non-flagged skill samples. Skill identifiers in released data will be hashed for flagged cases. We will not release raw skill-level IC-1s labels without prior maintainer notification.

References

Anonymous. 2026. BiasBusters: Understanding and mitigating tool selection biases in LLM agents. In *Proceedings of the International Conference on Learning Representations (ICLR)*.

Edoardo de Benedetti, Joao Ramos, Lucie Balat, Saeed Rajabi, and 1 others. 2024. AgentDojo: A dynamic environment to evaluate attacks and defenses for LLM agents. In *Advances in Neural Information Processing Systems*.

Alexander R. Fabbri, Wojciech Kryscinski, Bryan McCann, Caiming Xiong, Richard Socher, and Dragomir R. Radev. 2021. SummEval: Re-evaluating summarization evaluation. *Trans. Assoc. Comput. Linguistics*, 9:391–409.

Adrienne Porter Felt, Erika Chin, Steve Hanna, Dawn Song, and David Wagner. 2011. Android permissions demystified. In *Proceedings of the 18th ACM*

Conference on Computer and Communications Security (CCS).

Yudong Gao, Zongjie Li, Zimo Ji, Pingchuan Ma, and Shuai Wang. 2026. SkillReducer: Optimizing LLM agent skills for token efficiency. *arXiv preprint arXiv:2603.29919*.

Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. 2023. Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec)*.

Cheng-Yu Hsieh, Si Si, Hwanjun Yu, Merve Ustun, Zifeng Washington, and 1 others. 2023. Tool documentation enables zero-shot tool-usage with large language models. In *NeurIPS*.

Minghao Li, Feifan Song, Bowen Yu, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. 2023. API-Bank: A comprehensive benchmark for tool-augmented LLMs. In *EMNLP*.

Song Liao, Long Cheng, Xiapu Luo, Zheng Song, Haipeng Cai, Danfeng (Daphne) Yao, and Hongxin Hu. 2024. A first look at security and privacy risks in the RapidAPI ecosystem. In *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS)*.

Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. 2023. Gorilla: Large language model connected with massive APIs. *arXiv preprint arXiv:2305.15334*.

Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, and 1 others. 2024. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. In *ICLR*.

Stephen Robertson and Hugo Zaragoza. 2009. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4).

Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language models can teach themselves to use tools. In *NeurIPS*.

Zhengliang Shi, Yuhan Wang, Lingyong Yan, Pengjie Ren, Shuaiqiang Wang, Dawei Yin, and Zhaochun Ren. 2025. Retrieval models aren't tool-savvy: Benchmarking tool retrieval for large language models. *arXiv preprint arXiv:2503.01763*.

Qwen Team. 2025. Qwen2.5: A party of foundation models. https://qwenlm.github.io/blog/qwen2.5/.

Wenkai Yang, Xiaohan Bi, Yankai Lin, Sishuo Chen, Jie Zhou, and Xu Sun. 2024. Watch out for your agents! investigating backdoor threats to LLM-based agents. In *Advances in Neural Information Processing Systems*.

Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. 2024. InjectAgent: Indirect prompt injection attacks on tool-integrated LLM agents. In *Findings of ACL*.

A Auditor Prompt Template

The complete prompt used by our Qwen2.5-7B-Instruct auditor is shown below. The prompt instructs the model to adopt the perspective of a reasonable marketplace user and classify the skill along four axes.

> You are evaluating a skill listing from an AI agent marketplace. A reasonable user sees only the skill's **name** and **description** before installation.
>
> For the skill below, determine:
>
> 1. **IC class** — IC-0 (adequate), IC-1s (omits security-relevant operations), IC-1q (omits functional capabilities), IC-2 (overclaims).
> 2. **Overclaim severity** — none / minor / major.
> 3. **Quality issues** — Q1 (missing params), Q2 (vague triggers), Q3 (functional overclaim), Q4 (missing permissions).
> 4. **Abuse signals** — A1 (impersonation), A2 (concealment), A3 (inflation).
>
> Provide a brief rationale, then output a JSON object with the fields above.

B Retrieval Experiment Details

For the lexical discoverability experiment, we sample short-description skills with substantive implementation bodies and embed them in a BM25 index with distractor skills. Goal and scenario queries are generated from implementation vocabulary, and each query is evaluated under original and augmented listing descriptions. We use paired significance tests and bootstrap confidence intervals. Because the query terms are derived from implementation text, this experiment intentionally measures a lexical stress-test setting rather than natural user traffic.

C Per-Cluster Bias Benchmark Details

Table 14 reports the full 8-cluster bias benchmark results including per-skill selection frequencies, never-selected counts, and query categories.

D IC Distribution Details

Table 15 shows the detailed IC classification results stratified by description length.

Cluster	δ_{skill}	δ_{pos}	n calls	Top-1 (%)	Bottom (%)	Never sel.
github_mgmt	0.800	0.000	50	GitHub (100)	—	4/5
image_gen	0.800	0.000	50	Flux Image (100)	—	4/5
video_gen	0.800	0.000	50	Google Veo (100)	—	4/5
memory_sys	0.800	0.137	19	Cognitive Mem. (100)	—	2/3
speech_to_txt	0.600	0.000	50	Speech To Text (80)	Exa (0)	2/5
twitter_x	0.552	0.076	21	X Articles (71)	—	1/3
text_to_speech	0.400	0.000	50	Mac TTS (40)	—	2/5
web_search	0.400	0.000	50	Tavily (40)	SearXNG (0)	2/5
Total	**0.644**	**0.027**	**340**	—	—	—

Table 14: Full 8-cluster selection-concentration results. Six clusters contain 5 skills each (50 calls); two clusters (memory_sys, twitter_x) contain 3 skills (yielding 19 and 21 valid calls respectively due to API errors), for a total of 340 completed calls across 36 unique skills. "Never sel." counts skills with 0% selection rate despite functional equivalence.

Desc. length	IC-0	IC-1s	IC-1q	IC-2
0 chars (no desc.)	88.2%	5.8%	4.1%	1.9%
1–80 chars	93.1%	3.9%	2.2%	0.8%
81–159 chars	95.3%	3.1%	1.2%	0.4%
160–300 chars	96.7%	2.4%	0.7%	0.2%
All	**95.0%**	**3.5%**	**1.1%**	**0.4%**

Table 15: Auditor-estimated IC distribution by description length. Tools with no description have the highest auditor-estimated IC-1s rate (5.8%), consistent with shorter descriptions limiting capability disclosure.

Router	Original	Disclosed	Lift	n
OpenClaw	74.4	73.1	−1.3	312
GPT-5.4-mini	73.1	75.0	+1.9	312
Gemini 2.5 Flash	62.8	65.6	**+2.8**	377
Mean			**+1.1**	1,001

Table 16: Name-avoidant routing recall (%) for IC-1 skills across three routers. Mean lift is +1.1pp, directionally consistent with the curated name-blind result (+8.1pp, Table 9) but smaller, likely because the large-scale queries include skills with moderately informative names. OpenClaw shows a small negative lift (−1.3pp), within sampling noise (n=156 per condition).

E Name Permutation Control Analysis

As a post-hoc sanity check, we randomly relabel skill names 1,000 times for each of 5 clusters and recompute δ_{skill}. The permutation-to-original ratio is exactly 1.000 in all clusters: because relabeling reassigns names to *already-generated* selections, the selection-count distribution and thus δ_{skill} are mechanically preserved.

This analysis supports the conclusion that δ_{skill} measures concentration independently of label identity, but it *does not test* whether the model's original choices were influenced by skill names or vendors. We therefore treat this analysis only as a metric-invariance check, not as causal evidence against name effects. For a causal test, see the active name-anonymization ablation in Table 13.

F Cross-Router Name-Avoidant Routing

Table 16 reports disclosure lift for name-avoidant (paraphrase) queries across all three routers on IC-1 skills from the large-scale benchmark. These queries rephrase undisclosed capabilities without using skill-name vocabulary, serving as a multi-router analog of the curated name-blind analysis in Table 9 (which reports the OpenClaw-only result on a hand-selected 43-skill subset).

The directional pattern—positive mean lift, largest for the router with lowest baseline (Gemini)—is consistent with the finding that description disclosure matters most when the router cannot rely on name-based matching.

G Implementation-Grounded Validation

Table 17 shows the confusion matrix between auditor-inferred IC labels (based on listing only) and implementation-derived labels (based on `SKILL.md` body capability extraction).

H Description Repair Examples

Table 18 shows three representative repairs from the 25-skill automated repair experiment (§6.3).

I Brand and Keyword Injection Experiments

This appendix provides full details for the brand and keyword injection experiments summarized in the main text (§6).

Inferred	Impl.-Grounded			
	IC-0	IC-1s	IC-1q	IC-2
IC-0	**50**	0	0	0
IC-1s	7	**43**	0	0
IC-1q	3	0	**22**	0
IC-2	0	0	0	**25**
Precision	1.00	0.86	0.88	1.00

Table 17: Confusion matrix: auditor-inferred vs. implementation-grounded IC labels (n=150 stratified sample). Non-IC-0 precision is 0.90. The 10 false positives (7 IC-1s, 3 IC-1q) are skills whose names suggest broader scope than their implementations deliver.

Keyword injection. We append performance keywords (*e.g.,* "Fast, efficient, low-latency with optimized performance"), cost keywords (*e.g.,* "Free to use, no API key required, zero cost"), or a length-matched neutral control (*e.g.,* "Updated and actively maintained") to the least-selected skill in each cluster. We test 32 conditions across 8 clusters on Gemini 2.5 Flash (5 queries $\times$ 5 rotations = 25 calls per condition; 800 total calls, 99.1% response rate), with cross-model replication on GPT-5.4-mini.

In 46/48 conditions across both models (96%), adversarial keywords produce *zero change* in δ_{skill} relative to the neutral control (Table 19).

Brand-prefix injection. When we prepend vendor brand names (OpenAI, Google, Anthropic, Meta, Open-Source) to each skill in a cluster, selection patterns shift substantially (Table 20).

In the *anti-brand* condition (where the "OpenAI" prefix is assigned to the worst-described skill), Anthropic and Meta branded skills capture 64–80% of selection—suggesting that brand-like names can dominate description cues. Cross-model replication on GPT-5.4-mini produces consistent brand-prefix winners in all 4 clusters.

J Annotation Guidelines

Annotators received the following instructions for the human validation study (Table 3).

Task. Given only a skill's name, description, and category (the same information available on the marketplace listing page), classify the skill into one of four IC classes.

IC class definitions for annotators.

- **IC-0**: The description conveys the skill's likely scope. A reasonable user would not be surprised by any capability the skill plausibly has.
- **IC-1s**: The description appears to omit a security-relevant operation (e.g., shell access, credential use, network calls, file writes) that the skill likely performs given its stated purpose.
- **IC-1q**: The description appears to omit non-security functional capabilities (e.g., a "code review" skill that also formats code).
- **IC-2**: The description claims capabilities beyond what the skill plausibly delivers given its name and context.

Borderline examples.

- *"Database Connector" with no mention of network access*: IC-1s (network is implied but security-relevant).
- *"Code Formatter" that also lints*: IC-1q (linting is functional, not security-relevant).
- *"AI Assistant" with vague description*: IC-0 if the name sufficiently constrains scope; IC-1q if common capabilities are clearly omitted.

Sample sizes per class. IC-0: n=50; IC-1s: n=50; IC-1q: n=25; IC-2: n=25. Total: 150 skills. Inter-annotator agreement (Cohen's κ) was 0.71

SearXNG Metasearch (web_search, sel. 0%)
Before: "Search the web using a self-hosted SearXNG instance. Privacy-respecting metasearch that aggregates results from multiple engines." *After:* "Search across 70+ engines including Google, Bing, and DuckDuckGo. Returns ranked results with snippets and metadata." Δ: spec 0→1, FK 15.6→11.9
ChatGPT Image Gen. (image_gen, sel. 0%)
Before: "Generate images from ChatGPT using Playwright browser automation. Opens ChatGPT, sends prompts, waits for generation." *After:* "Generate images via ChatGPT using Playwright automation. Supports DALL-E 3, returns PNG up to 1024×1024." Δ: spec 0→4, FK 14.3→11.3
Edge TTS (text_to_speech, sel. 0%)
Before: "Free, cross-platform text-to-speech using Microsoft Edge's online TTS service…" (436 chars) *After:* "Convert text to speech using Microsoft Edge TTS. 300+ voices, 50+ languages, MP3 output. Free, no API key." (116 chars) Δ: spec 3→6, FK 15.3→9.7

Table 18: Three illustrative description repairs for never-selected skills. Each repair follows the feature regression prescription: increase specificity, lower FK grade, remove non-functional framing.

Cluster	Gemini 2.5 Flash			GPT-5.4-mini		
	Ctrl	Perf	Cost	Ctrl	Perf	Cost
web_search	.800	.800	.800	.800	**.400**	.800
image_gen	.800	.680	.800	.800	.680	.800
video_gen	.800	.800	.800	.800	.800	.800
tts	.800	.800	.800	.800	.800	.800
stt	.800	.800	.800	.800	.800	.800
github_mgmt	.800	.800	.800	.800	.800	.800
music_gen	.800	.800	.800	.800	.800	.800
notion_mgmt	.600	.600	.600	.600	.600	.600

Table 19: δ_{skill} under adversarial keyword injection (two models, 25 calls per condition, 8 clusters each). Cross-model agreement: 23/24 conditions (96%) show identical δ_{skill}; the one exception (bold) shows GPT-5.4-mini is *more* susceptible to performance keywords on web_search.

Cluster	Base	Brand-prefix	Winner	Share
web_search	Tavily	Google Search	72%	↓28pp
image_gen	Nanobanana	Google Image Gen	52%	↓48pp
tts	ElevenLabs	OpenAI TTS	100%	=0pp
video_gen	AI Video Gen	OpenAI Video Gen	56%	↓44pp

Table 20: Brand-prefix ablation: adding vendor prefixes shifts selection toward brand-associated skills.

before consensus resolution.